

Repository Code Listing

ChinesePostman.h

```
1  #include "../Minimum-Cost-Perfect-Matching/Graph.h"
2  #include "../Minimum-Cost-Perfect-Matching/Matching.h"
3  #include "../Dijkstra.h"
4
5  bool Connected(const Graph &G) {
6      vector<bool> visited(G.GetNumVertices(), false);
7      list<int> L;
8
9      int n = 0;
10     L.push_back(0);
11     while (not L.empty()) {
12         int u = L.back();
13         L.pop_back();
14         if (visited[u])
15             continue;
16
17         visited[u] = true;
18         n++;
19
20         for (list<int>::const_iterator it = G.AdjList(u).begin();
21              it != G.AdjList(u).end(); it++) {
22             int v = *it;
23             L.push_back(v);
24         }
25     }
26
27     return G.GetNumVertices() == n;
28 }
29
30 /*
31  Solves the chinese postman problem
32  returns a pair containing a list and a double
33  the list is the sequence of vertices in the solution
34  the double is the solution cost
35  */
36 pair<list<int>, double> ChinesePostman(const Graph &G,
37                                         const vector<double> &cost) {
38     // Check if the graph is connected
39     if (not Connected(G))
40         throw "Error: Graph is not connected";
41
42     // Build adjacency lists using edges in the graph
43     vector<list<int>> A(G.GetNumVertices(), list<int>());
44     for (int u = 0; u < G.GetNumVertices(); u++)
45         A[u] = G.AdjList(u);
46
47     // Find vertices with odd degree
48     vector<int> odd;
49     for (int u = 0; u < G.GetNumVertices(); u++)
50         if (A[u].size() % 2)
51             odd.push_back(u);
52
53     // If there are odd degree vertices
54     if (not odd.empty()) {
```

```

55 // Create a graph with the odd degree vertices
56 Graph O(odd.size());
57 for (int u = 0; u < (int)odd.size(); u++)
58     for (int v = u + 1; v < (int)odd.size(); v++)
59         O.AddEdge(u, v);
60
61 vector<double> costO(O.GetNumEdges());
62
63 // Find the shortest paths between all odd degree vertices
64 vector<vector<int>> shortestPath(O.GetNumVertices());
65 for (int u = 0; u < (int)odd.size(); u++) {
66     pair<vector<int>, vector<double>> sp = Dijkstra(G, odd[u], cost);
67
68     shortestPath[u] = sp.first;
69
70     // The cost of an edge uv in O will be the cost of the
71 corresponding
72 // shortest path in G
73     for (int v = 0; v < (int)odd.size(); v++)
74         if (v != u)
75             costO[O.GetEdgeIndex(u, v)] = sp.second[odd[v]];
76 }
77
78 // Find the minimum cost perfect matching of the graph of the odd
79 degree
80 // vertices
81 Matching M(O);
82 pair<list<int>, double> p =
83 M.SolveMinimumCostPerfectMatching(costO);
84 list<int> matching = p.first;
85
86 // If an edge uv is in the matching, the edges in the shortest path
87 from u
88 // to v should be duplicated in G
89 for (list<int>::iterator it = matching.begin(); it !=
90 matching.end();
91     it++) {
92     pair<int, int> p = O.GetEdge(*it);
93     int u = p.first, v = odd[p.second];
94
95     // Go through the path adding the edges
96     int w = shortestPath[u][v];
97     while (w != -1) {
98         A[w].push_back(v);
99         A[v].push_back(w);
100         v = w;
101         w = shortestPath[u][v];
102     }
103 }
104 }
105
106 // Find an Eulerian cycle in the graph implied by A
107 list<int> cycle;
108 // This is to keep track of how many times we can traverse an edge
109 vector<int> traversed(G.GetNumEdges(), 0);
110 for (int u = 0; u < G.GetNumVertices(); u++) {
111     for (list<int>::iterator it = A[u].begin(); it != A[u].end(); it++)
112     {
113         int v = *it;
114
115         // we do this so that the edge is not counted twice
116         if (v < u)
117             continue;
118

```

```

        traversed[G.GetEdgeIndex(u, v)]++;
    }
}

119 cycle.push_back(0);
120 list<int>::iterator itp = cycle.begin();
121 double obj = 0;
122 while (itp != cycle.end()) {
123     // Let u be the current vertex in the cycle, starting at the first
124     int u = *itp;
125     list<int>::iterator jtp = itp;
126     jtp++;
127
128     // if there are non-traversed edges incident to u, find a subcycle
129     starting
130     // at u replace u in the cycle by the subcycle
131     while (not A[u].empty()) {
132         while (not A[u].empty() and
133             traversed[G.GetEdgeIndex(u, A[u].back())] == 0)
134             A[u].pop_back();
135
136         if (not A[u].empty()) {
137             int v = A[u].back();
138             A[u].pop_back();
139             cycle.insert(jtp, v);
140             traversed[G.GetEdgeIndex(u, v)]--;
141             obj += cost[G.GetEdgeIndex(u, v)];
142             u = v;
143         }
144     }
145
146     // go to the next vertex in the cycle and do the same
147     itp++;
148 }

    return pair<list<int>, double>(cycle, obj);
}

```

Dijkstra.h

```

1 #pragma once
2
3 #include "../Minimum-Cost-Perfect-Matching/BinaryHeap.h"
4 #include "../Minimum-Cost-Perfect-Matching/Globals.h"
5 #include "../Minimum-Cost-Perfect-Matching/Graph.h"
6 #include <limits>
7 using namespace std;
8
9 // Dijkstra's algorithm using binary heap
10 // Returns a pair (vector<int>, vector<double>)
11 // vector<double> gives the cost of the optimal path to each vertex
12 // vector<int> gives the parent of each vertex in the tree of optimal
13 paths
14 pair<vector<int>, vector<double>> Dijkstra(const Graph &G, int origin,
15                                           const vector<double> &cost) {
16     BinaryHeap B;
17
18     int n = G.GetNumVertices();
19
20     // Father of each vertex in the optimal path tree

```

```

21     vector<int> father(n, -1);
22     // Used to indicate whether a vertex is permanently labeled
23     vector<bool> permanent(n, false);
24
25     vector<double> pathCost(n, numeric_limits<double>::infinity());
26
27     // Put s in the heap
28     B.Insert(0, origin);
29     pathCost[origin] = 0;
30
31     for (int i = 0; i < n; i++) {
32         // Select the vertex that can be reached with smallest cost
33         int u = B.DeleteMin();
34
35         permanent[u] = true;
36
37         // Update the heap with vertices adjacent to u
38         for (list<int>::const_iterator it = G.AdjList(u).begin();
39              it != G.AdjList(u).end(); it++) {
40             int v = *it;
41
42             if (permanent[v])
43                 continue;
44
45             double c = pathCost[u] + cost[G.GetEdgeIndex(u, v)];
46
47             // v has not been discovered yet
48             if (father[v] == -1) {
49                 father[v] = u;
50                 pathCost[v] = c;
51                 B.Insert(c, v);
52             }
53             // we found a cheaper connection to v
54             else if (LESS(c, pathCost[v])) {
55                 father[v] = u;
56                 pathCost[v] = c;
57                 B.ChangeKey(c, v);
58             }
59         }
60     }
61
62     if (B.Size() > 0)
63         throw "Error: graph is not connected";
64
65     return make_pair(father, pathCost);
66 }

```

Example.cpp

```

1  #include "../Minimum-Cost-Perfect-Matching/Graph.h"
2  #include "ChinesePostman.h"
3  #include <fstream>
4  #include <iostream>
5  #include <sstream>
6  #include <string>
7  using namespace std;
8
9  pair<Graph, vector<double>> ReadWeightedGraph(string filename) {
10     // Please see Graph.h for a description of the interface

```

```

11
12 ifstream file;
13 file.open(filename.c_str());
14
15 string s;
16 getline(file, s);
17 stringstream ss(s);
18 int n;
19 ss >> n;
20 getline(file, s);
21 ss.str(s);
22 ss.clear();
23 int m;
24 ss >> m;
25
26 Graph G(n);
27 vector<double> cost(m);
28 for (int i = 0; i < m; i++) {
29     getline(file, s);
30     ss.str(s);
31     ss.clear();
32     int u, v;
33     double c;
34     ss >> u >> v >> c;
35
36     G.AddEdge(u, v);
37     cost[G.GetEdgeIndex(u, v)] = c;
38 }
39
40 file.close();
41 return make_pair(G, cost);
42 }
43
44 int main(int argc, char *argv[]) {
45     string filename = "";
46
47     int i = 1;
48     while (i < argc) {
49         string a(argv[i]);
50         if (a == "-f")
51             filename = argv[++i];
52         i++;
53     }
54
55     if (filename == "") {
56         cout << endl << "usage: ./example -f <filename>" << endl << endl;
57         cout << "-f followed by file name to specify the input file." <<
58 endl
59         << endl;
60
61         cout << "File format:" << endl;
62         cout << "The first two lines give n (number of vertices) and m
63 (number of "
64         "edges),"
65         << endl;
66         cout << "Each of the next m lines has a tuple (u, v [, c])
67 representing an "
68         "edge,"
69         << endl;
70         cout << "where u and v are the endpoints (0-based indexing) of the
71 edge "
72         "and c is its cost"
73         << endl;
74     return 1;

```

```

    }

75     try {
76         Graph G;
77         vector<double> cost;
78
79         // Read the graph
80         pair<Graph, vector<double>> p = ReadWeightedGraph(filename);
81         G = p.first;
82         cost = p.second;
83
84         // Solve the problem
85         pair<list<int>, double> sol = ChinesePostman(G, cost);
86
87         cout << "Solution cost: " << sol.second << endl;
88
89         list<int> s = sol.first;
90
91         // Print edges in the solution
92         cout << "Solution:" << endl;
93         for (list<int>::iterator it = s.begin(); it != s.end(); it++)
94             cout << *it << " ";
95         cout << endl;
96     } catch (const char *msg) {
97         cout << msg << endl;
98         return 1;
99     }
100
    return 0;
}

```